Spring 2025

Artificial Intelligence

Language Models & Transformers

Tianlong Chen

Donald Trump was the 45th president of the United States

Language models *(such as ChatGPT) just predict the next word likely to a given sequence of words*

Auto-complete on your phone message app does the same thing!

Learn to do this from billions of documents on the web







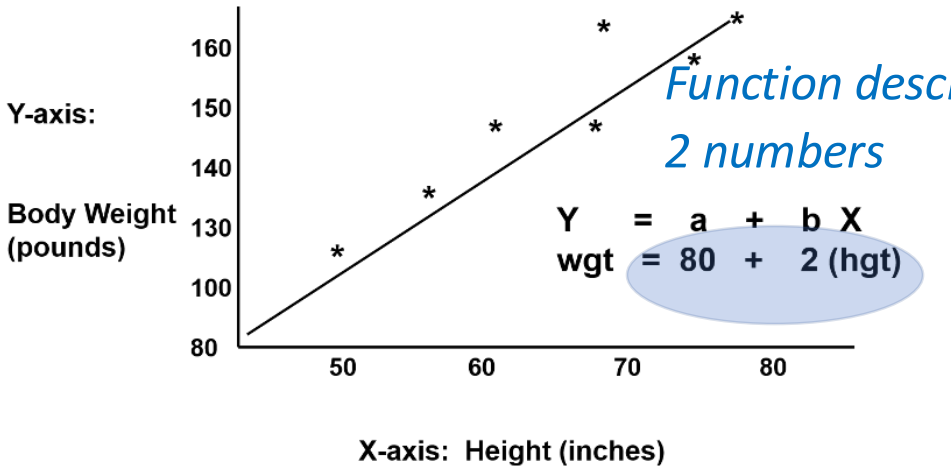
Just bigger! Better! Stronger!

Donald Trump was the 45th president of the United States

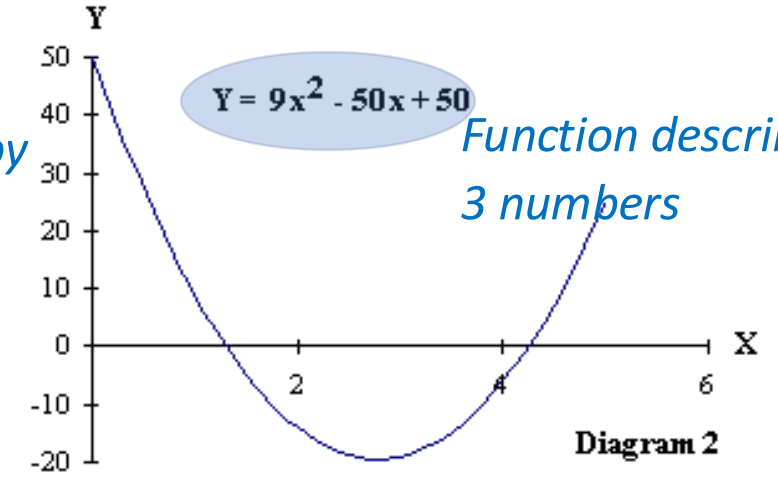
Justin Trudeau, the 23rd prime-minister of Canada

Can learn what words are similar, what words tend to occur together, what words follow each other …

What is ChatGPT?



Function described by 2 numbers



Function described by 3 numbers

ChatGPT is a complicated function described by about 175 billion numbers (or 'parameters')

What does it do?

T

write a biblical verse in the style of the King James Bible explaining how to remove a peanut butter sandwich from a VCR

And it came to pass that a man was troubled by a peanut butter sandwich, for it had been placed within his VCR, and he knew not how to remove it.

And he cried out to the Lord, saying, "Oh, Lord, how can I remove this sandwich from my VCR, for it is stuck fast and will not budge?"

And the Lord spoke unto him, saying, "Fear not, my child, for I shall guide thy hand and show thee the way. Take thy butter knife, and carefully insert it between the sandwich and the VCR, and gently pry them apart. And with patience and perseverance, the sandwich shall be removed, and thy VCR shall be saved."

GJ

write the code to check if Prisma package is installed in typescript

To check if the Prisma package is installed in the current project using TypeScript, you can use the `require` function in Node.js. If the package is installed and available to be required, `require` will return the exports of the package. If the package is not found, it will throw an error.

Here's an example of how you can check if the Prisma package is installed in TypeScript:

Copy code

```
try {
  const prisma: any = require('prisma');
  console.log('Prisma package found and required successfully');
} catch (error) {
  console.error('Prisma package not found:', error);
}
```

SS

Create a limerick about ChatGPT at a bar

At a bar, ChatGPT was quite the star
Sipping on drinks, like a superstar
People gathered around
To hear its wit astound
As it told limericks, both near and far

SS

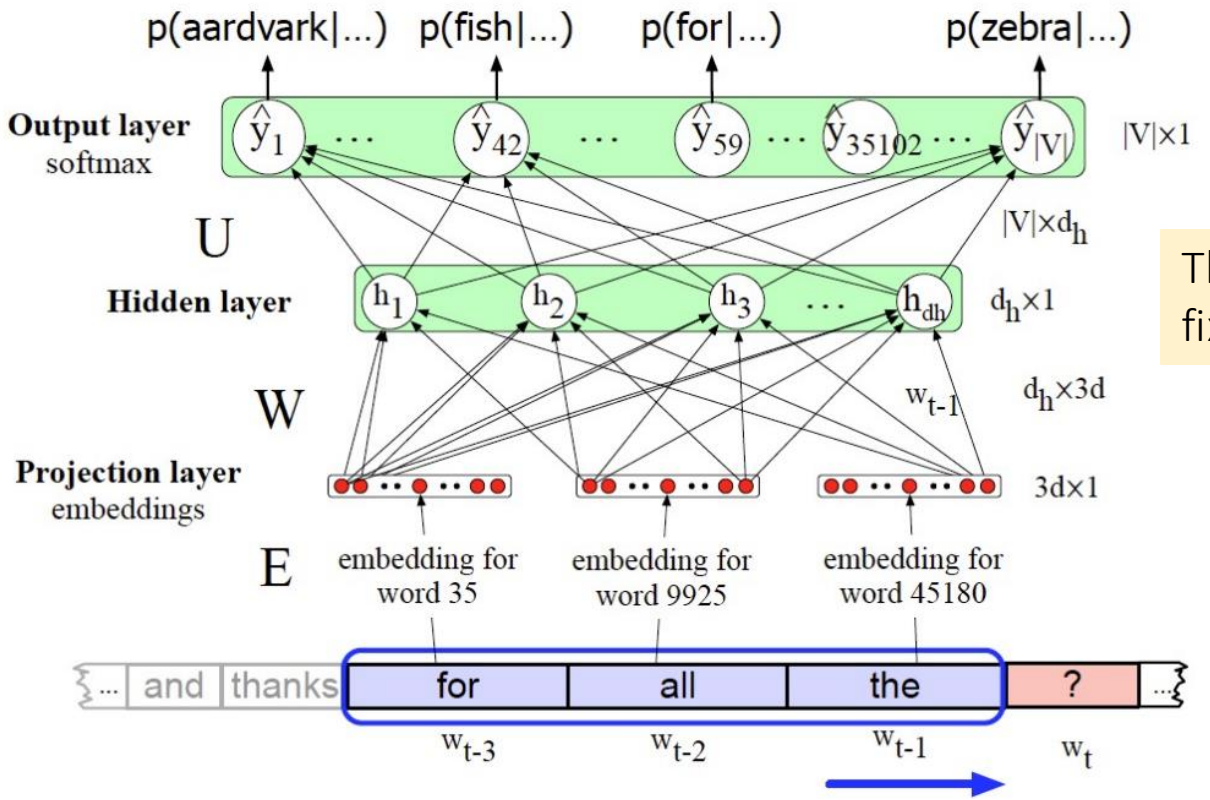
Generate a math-heavy assignment question about computing gradients in an RNN-based language model.

Here is a possible math-heavy assignment question:

Consider an RNN-based language model with a hidden state h_t at time step t, which is defined by the following update equation:

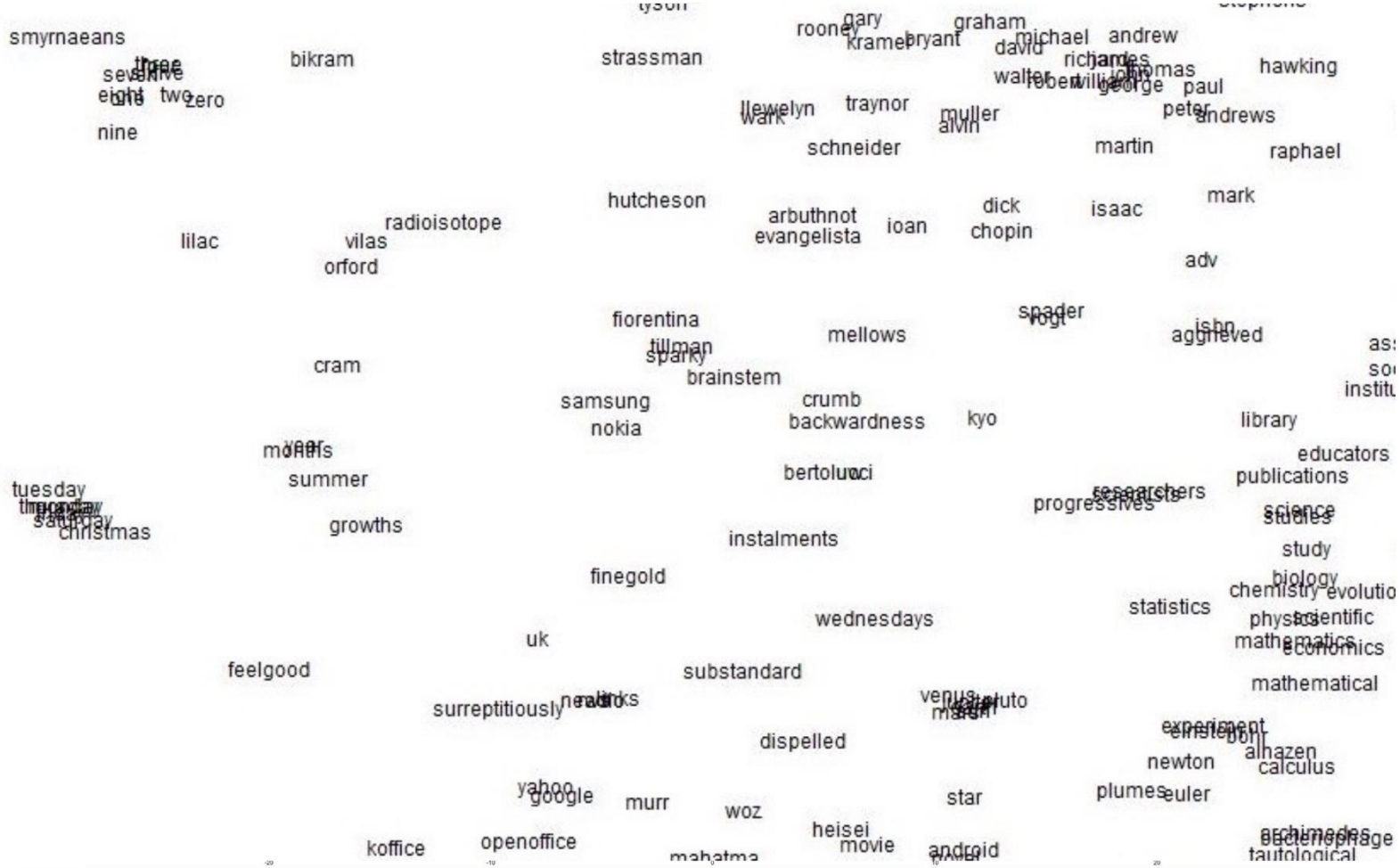
$$h_t = f(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$$

Language Modeling with FFNs?



This is modeling fixed-length context

Word embeddings



How it works?

GPT: Generative Pretrained Transformers

Generative

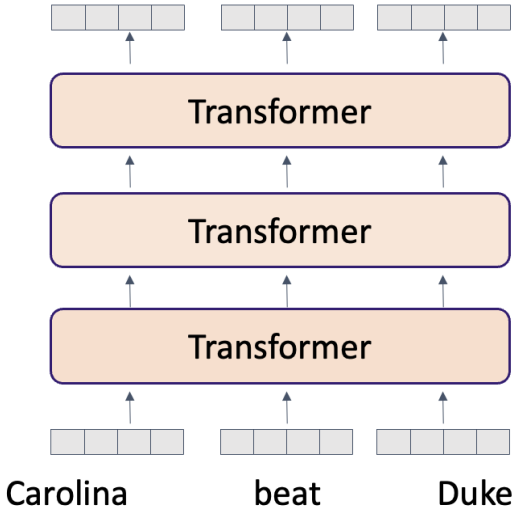
SS Create a limerick about ChatGPT at a bar

At a bar, ChatGPT was quite the star
Sipping on drinks, like a superstar
People gathered around
To hear its wit astound
As it told limericks, both near and far

Pretrained



Transformers



How it works?

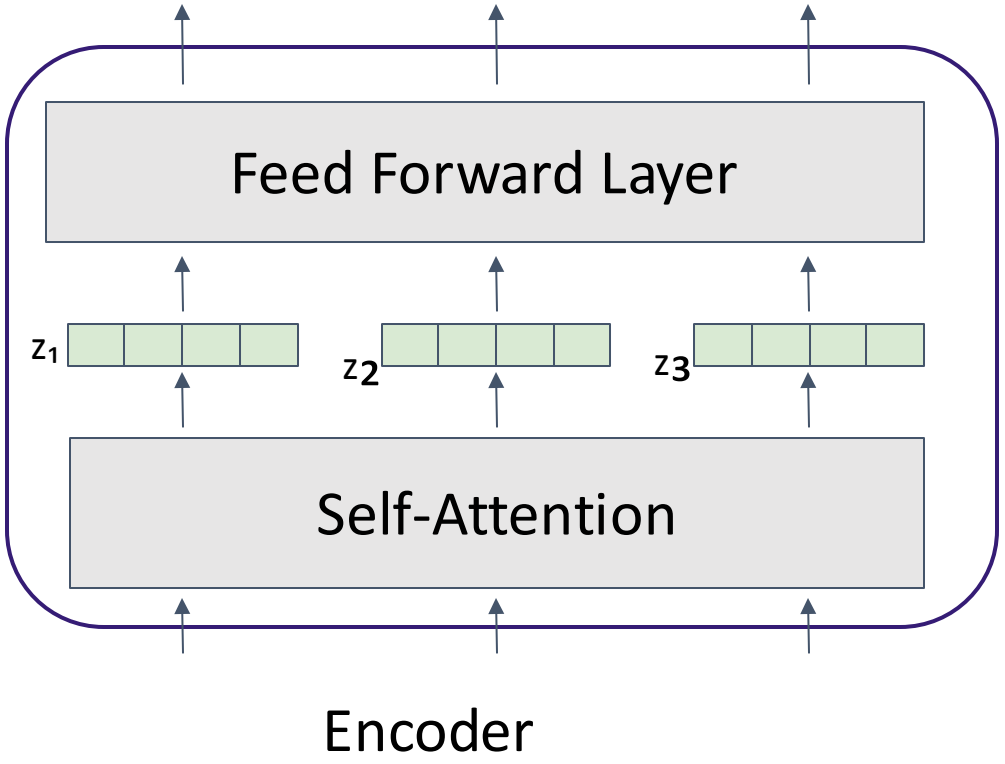
Intelligent behavior of language models is **emergent**, not pre-programmed

Neural network parameters (those 175 billion numbers!) are very hard to interpret

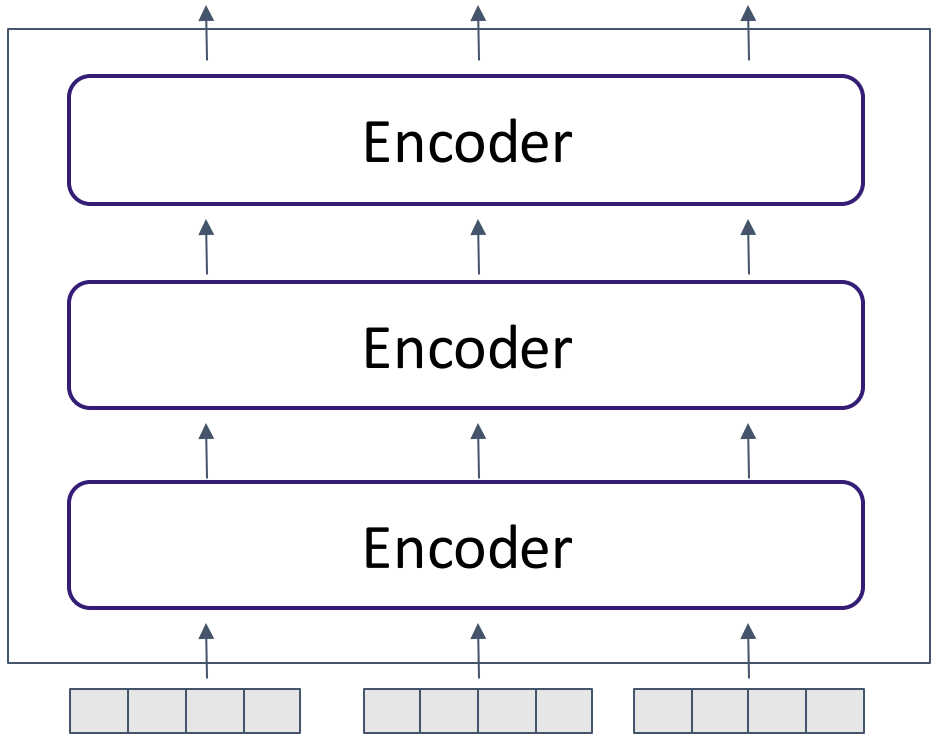
Actual answer: No one knows!



How to build it?

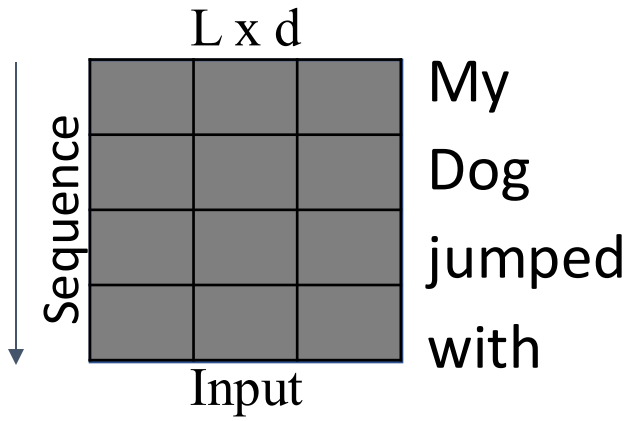


Encoder

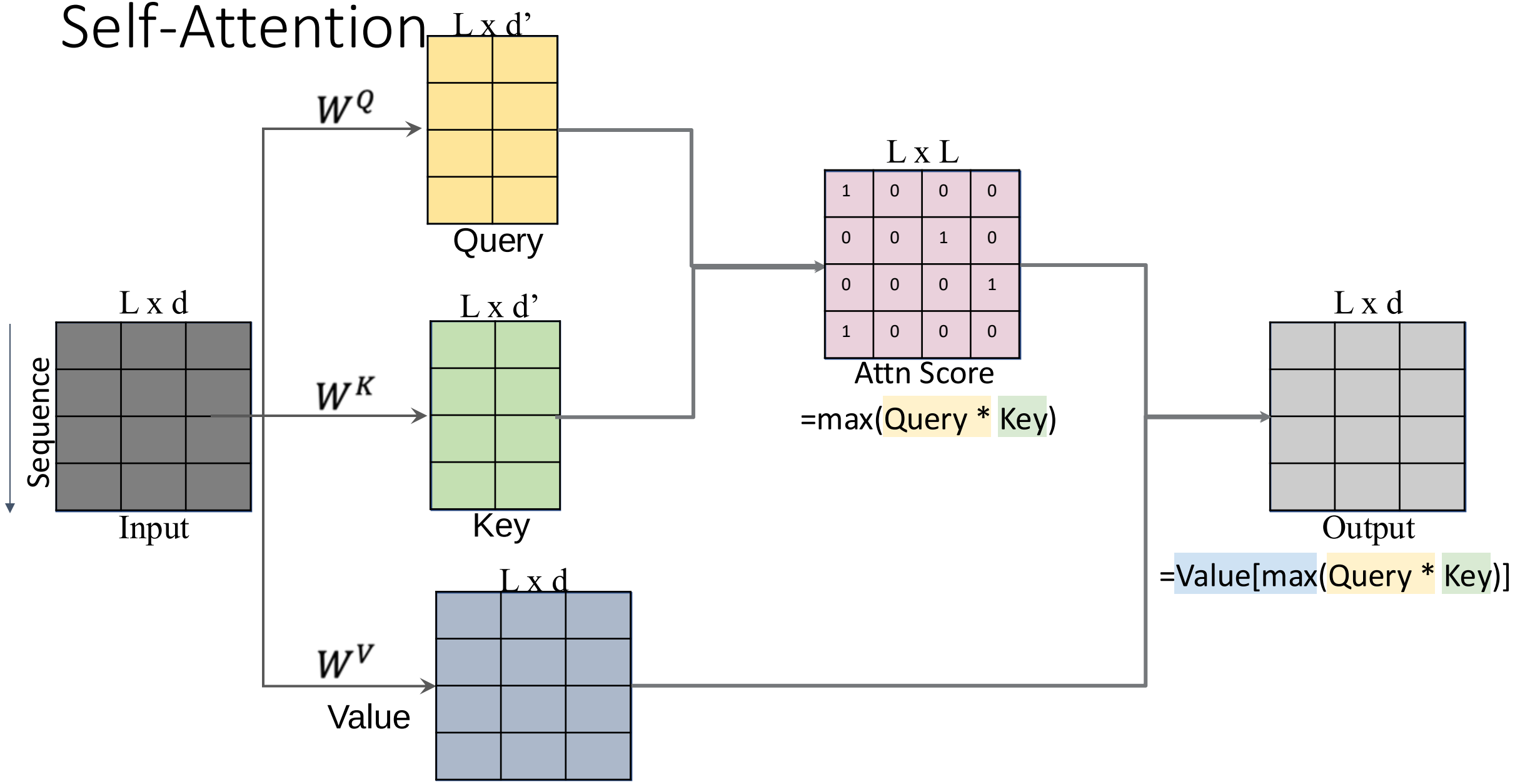


Transformer Encoder Stack

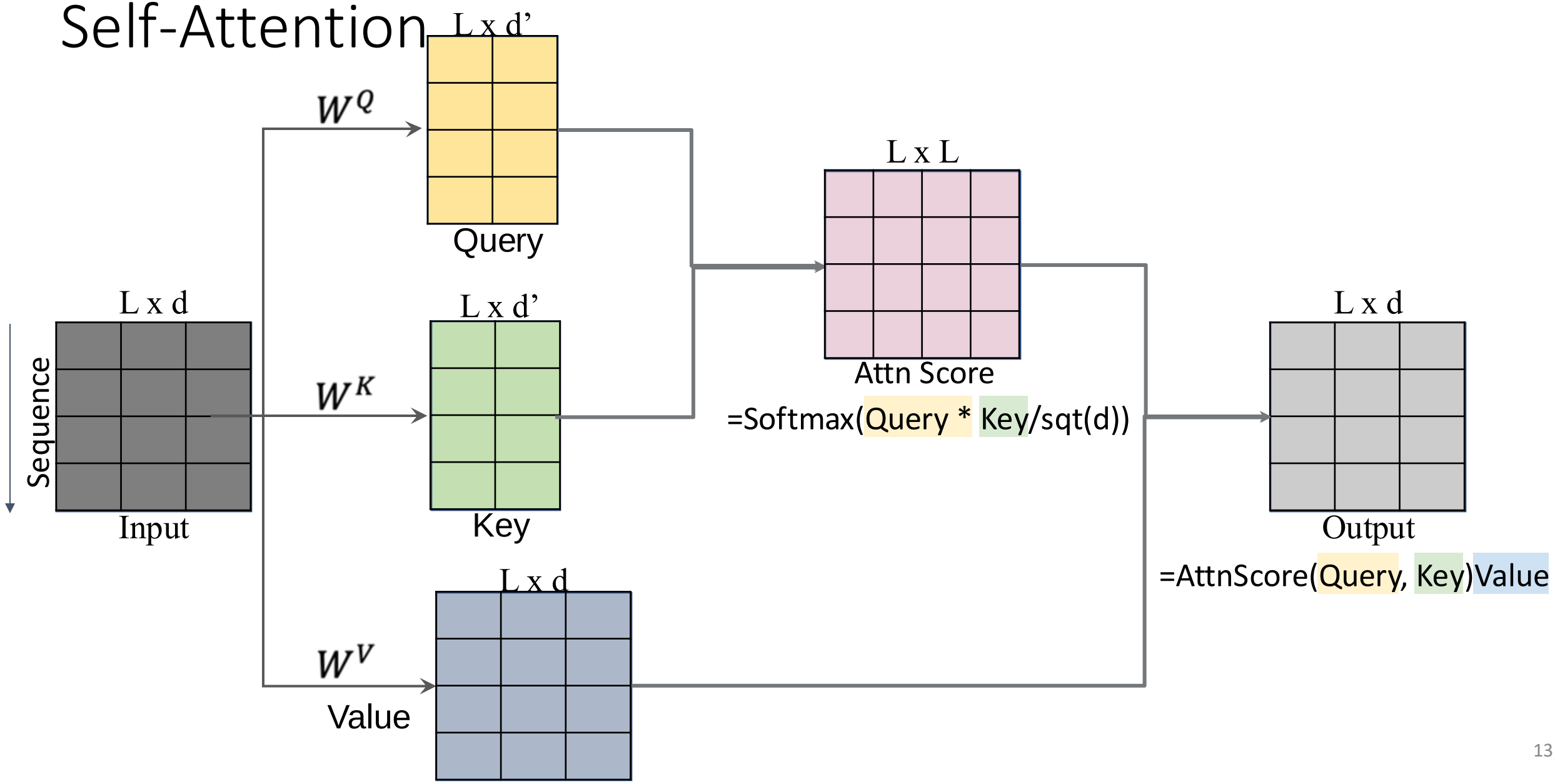
Self-Attention



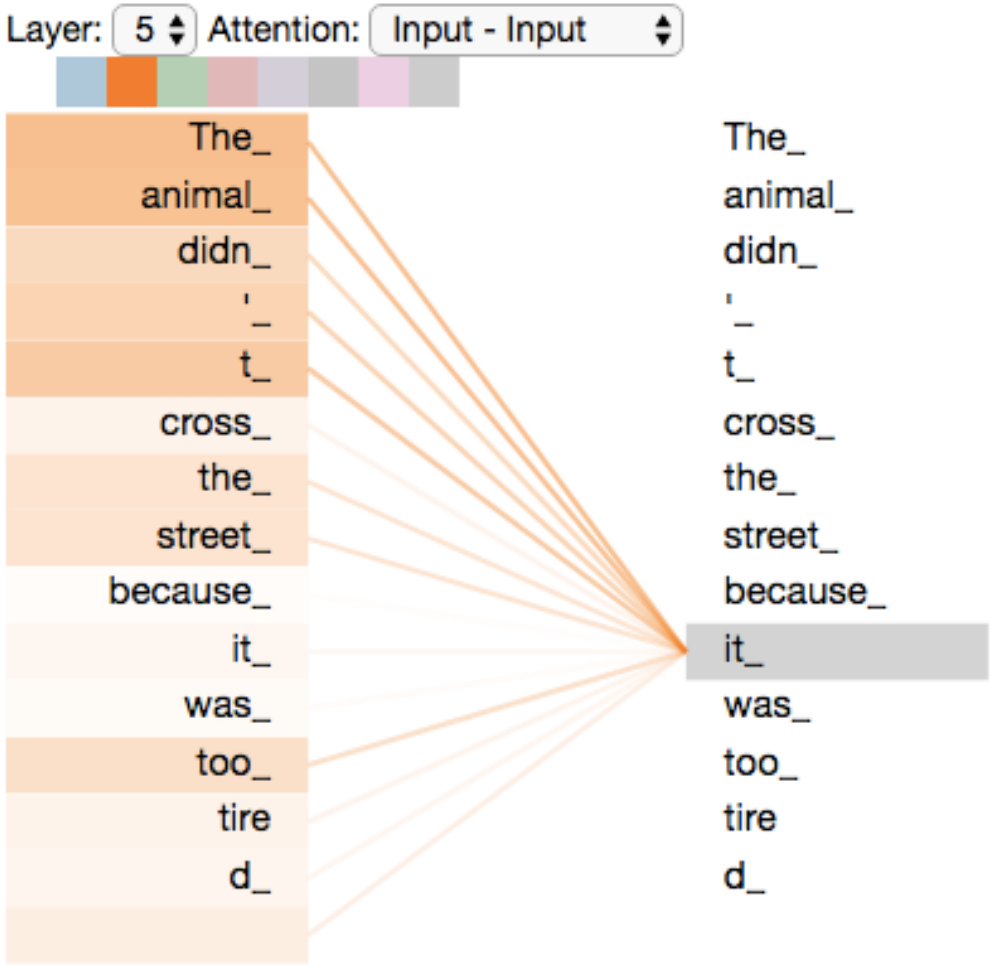
Self-Attention



Self-Attention



What does self attention do?



Attn Score

Image from https://jalammar.github.io/illustrated-transformer/

Multi-Head Attention

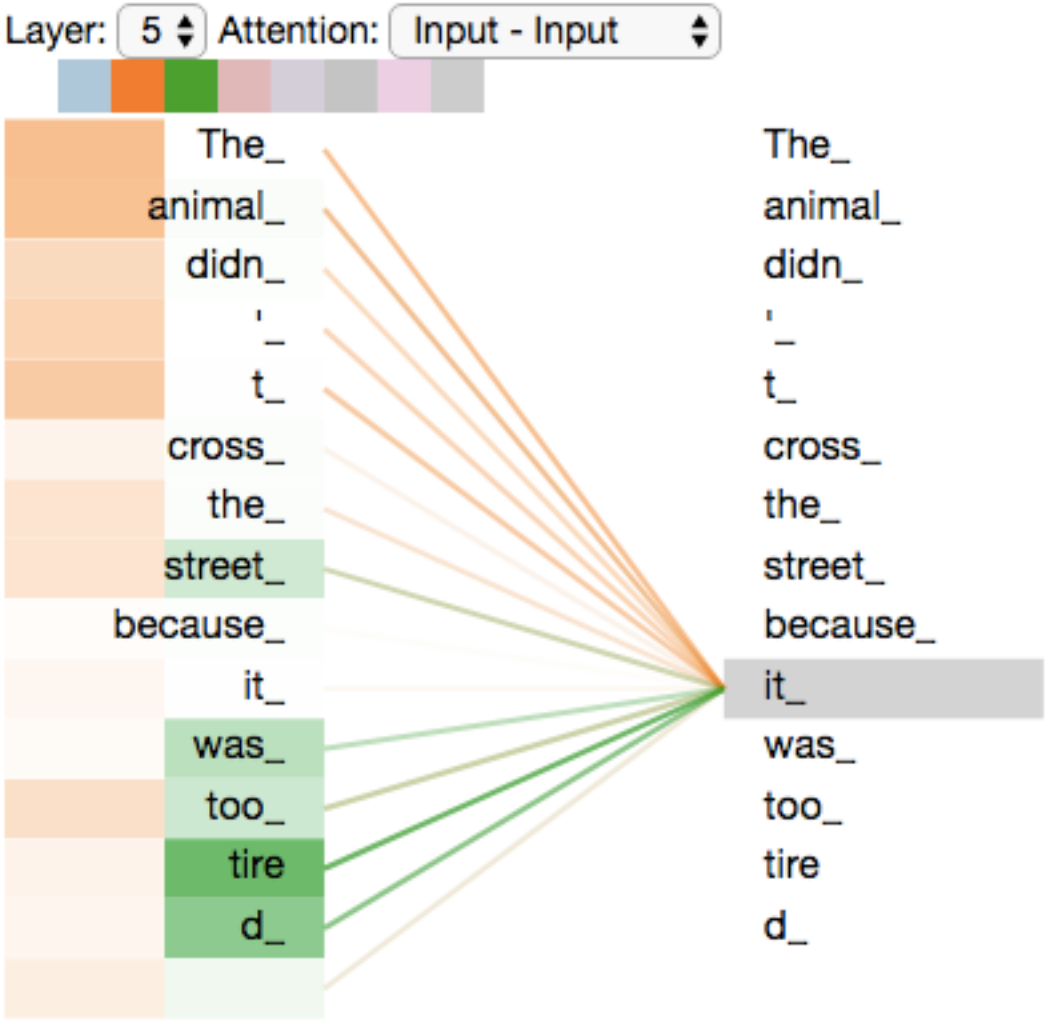


Image from https://jalammar.github.io/illustrated-transformer/

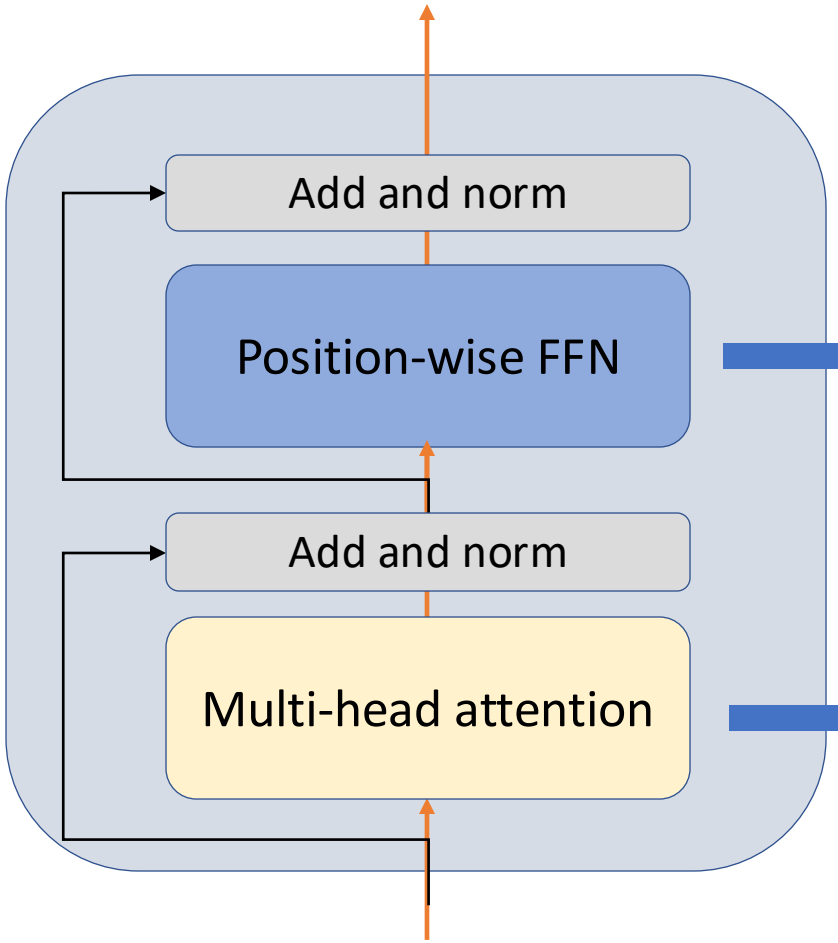
Feedforward Network (FFN)

- For each position in the sequence

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- W_1, W_2, b_1, b_2 are parameters of the feedforward network

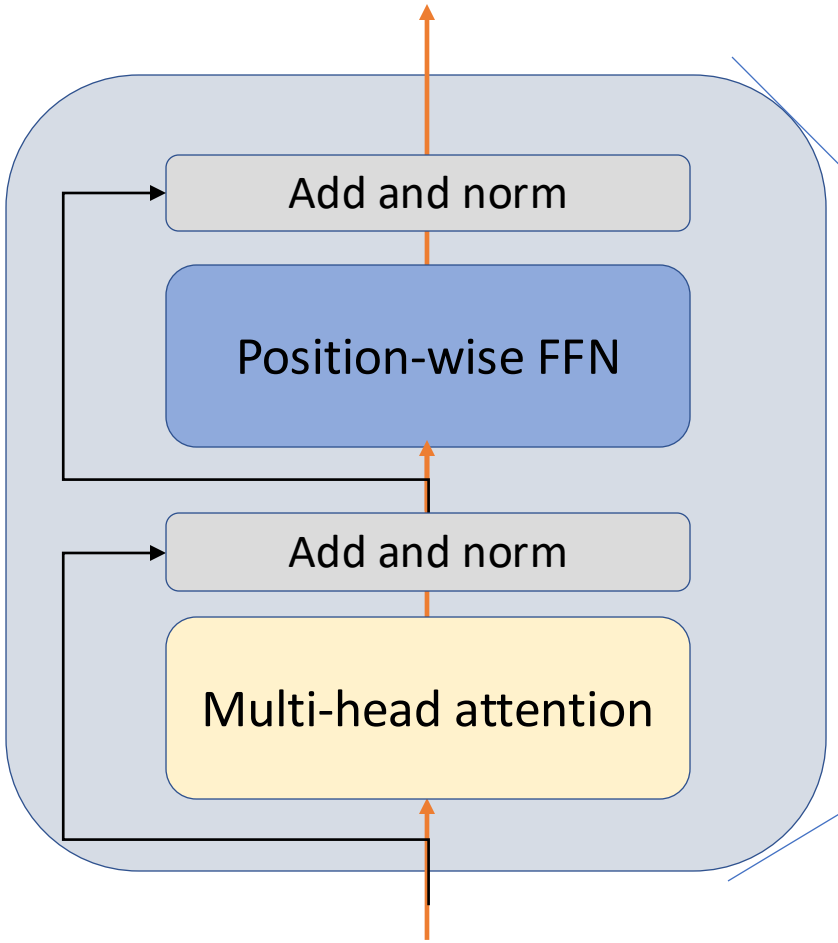
Parameters of a Transformer

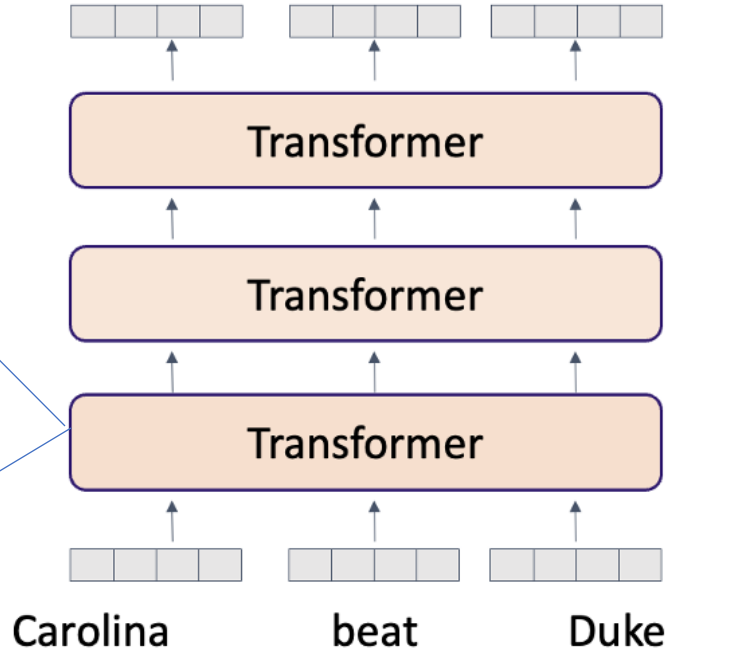


Parameters
W_1, W_2, b_1, b_2

For each head,
we have W^Q, W^K, W^V

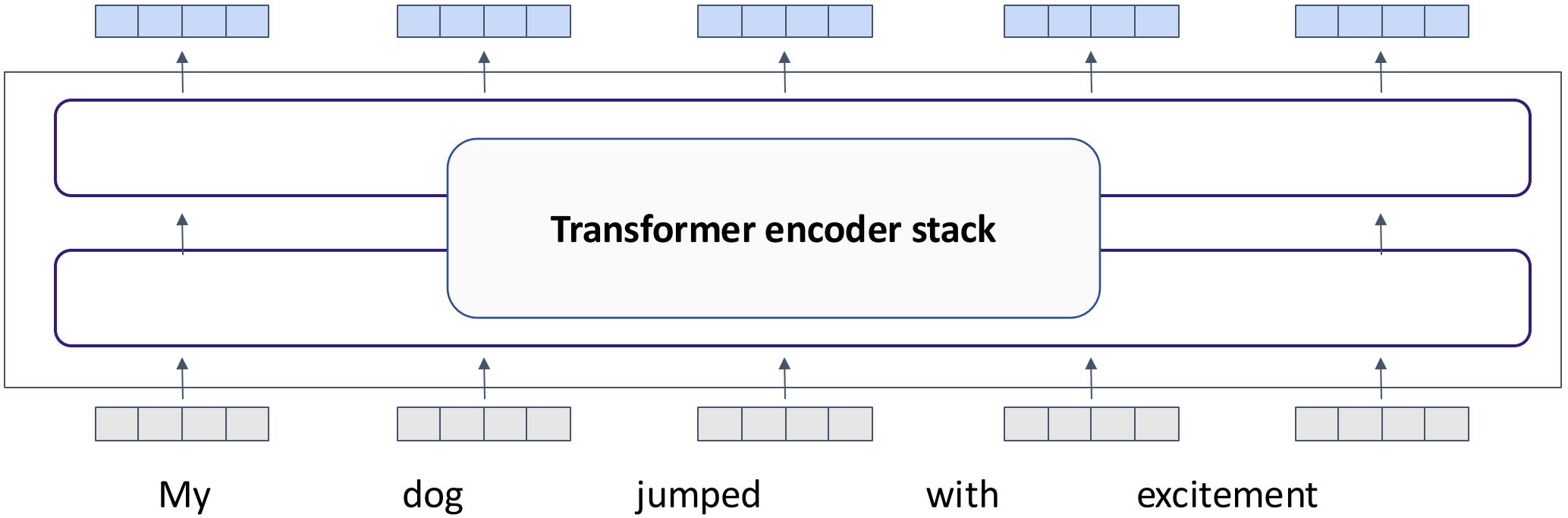
Multiple Layers!



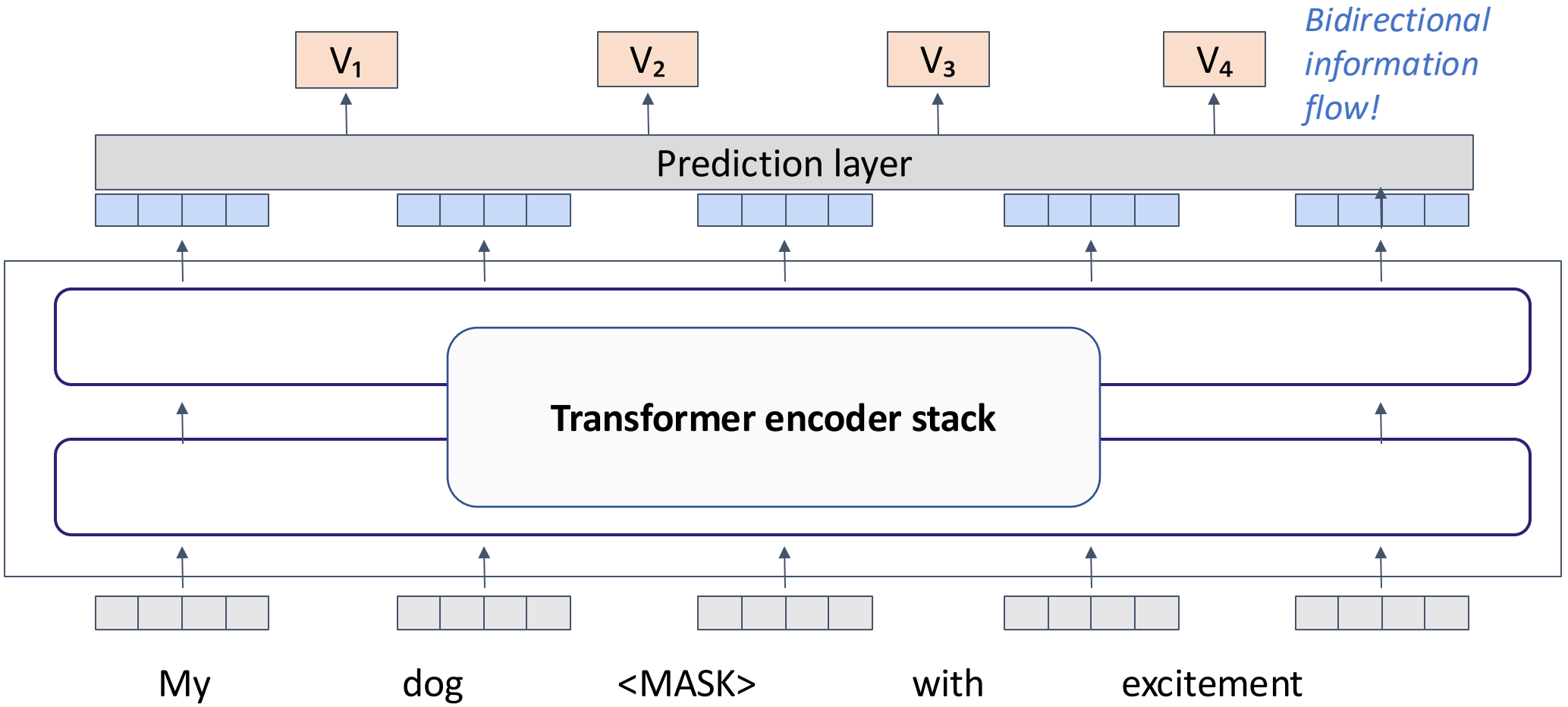


Embeddings are learnable too.

Pretraining and Fine-Tuning

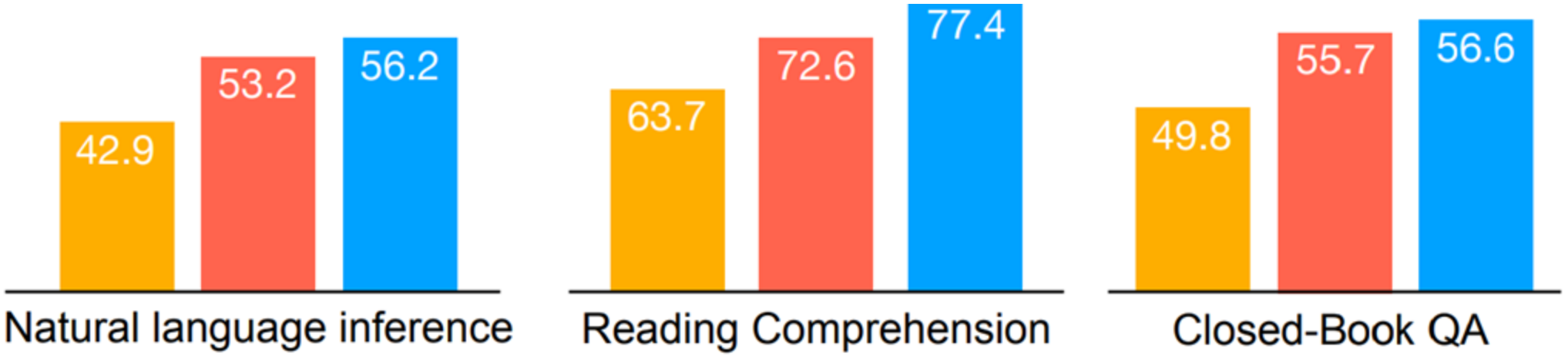


Pretraining and Fine-Tuning



Fine-Tuning

- Need small amount of data for downstream task for training
- Only one additional (final) layer is trained for specific tasks



Zero shot | Few shot | Full language model

Image from Fine-Tuned Language Models Are Zero-Shot Learners

Advanced Transformer Architectures

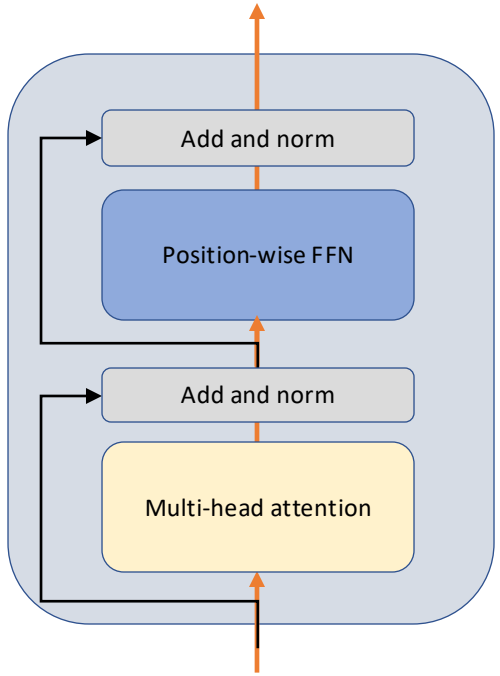
Outline

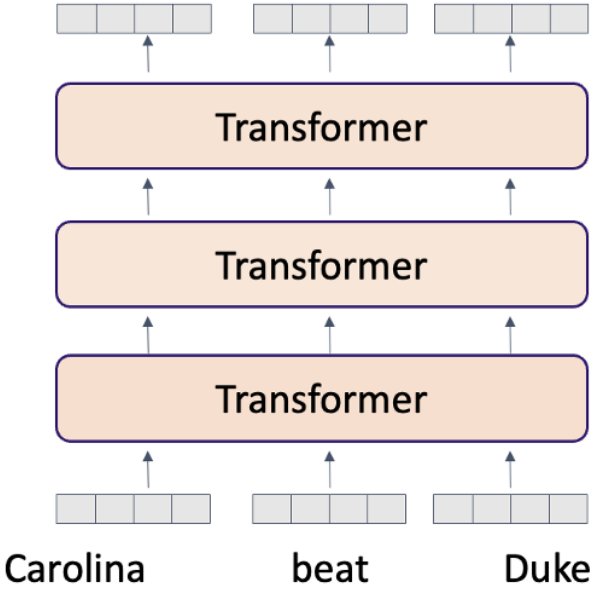
- Encoder only Transformers
- Decoder only Transformers
- Encoder-Decoder Transformers
- Future Directions & Challenges

Encoder-Only Transformers

Encoder-Only Transformers

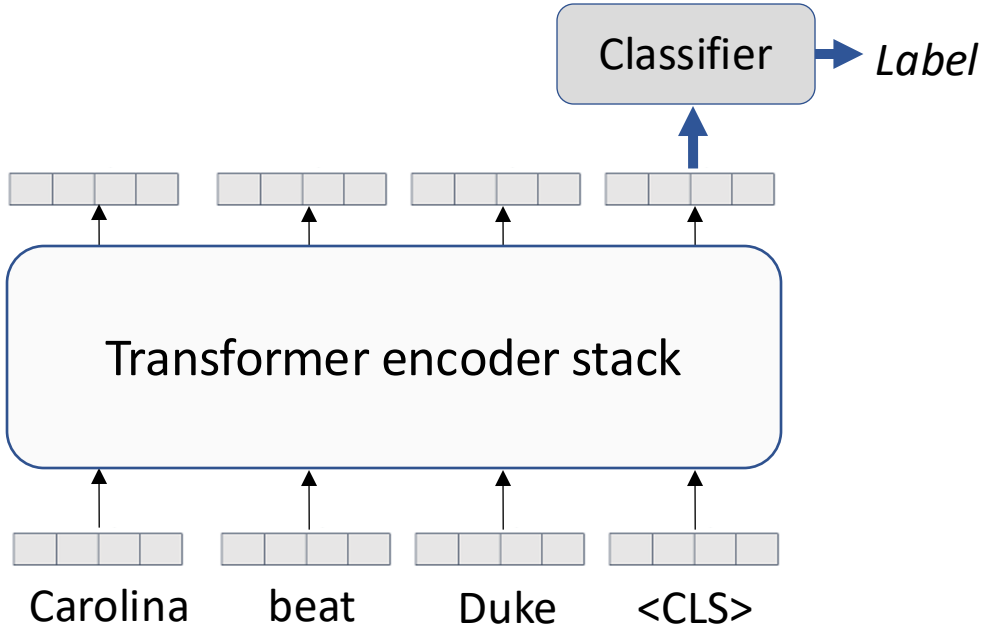
- Process input sequences without generating outputs
- Excel in tasks such as text classification and sentiment analysis



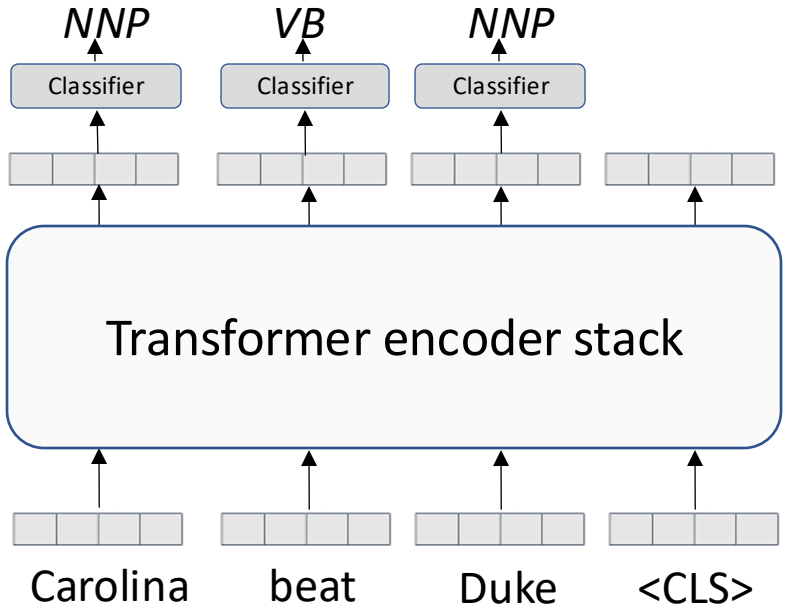


Encoder-Only Transformers (cont.)

- **Text classification**



- **Sequence labeling**



BERT (Bidirectional Encoder Representations From Transformers)

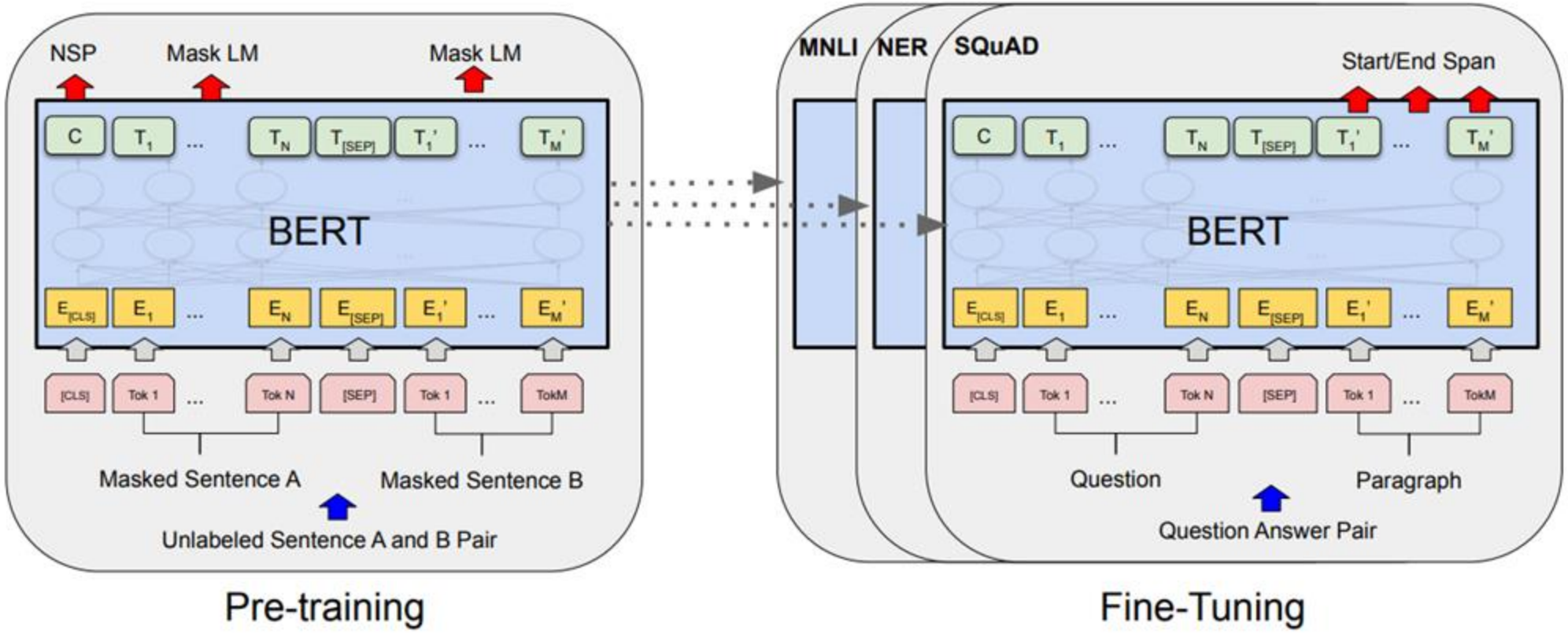
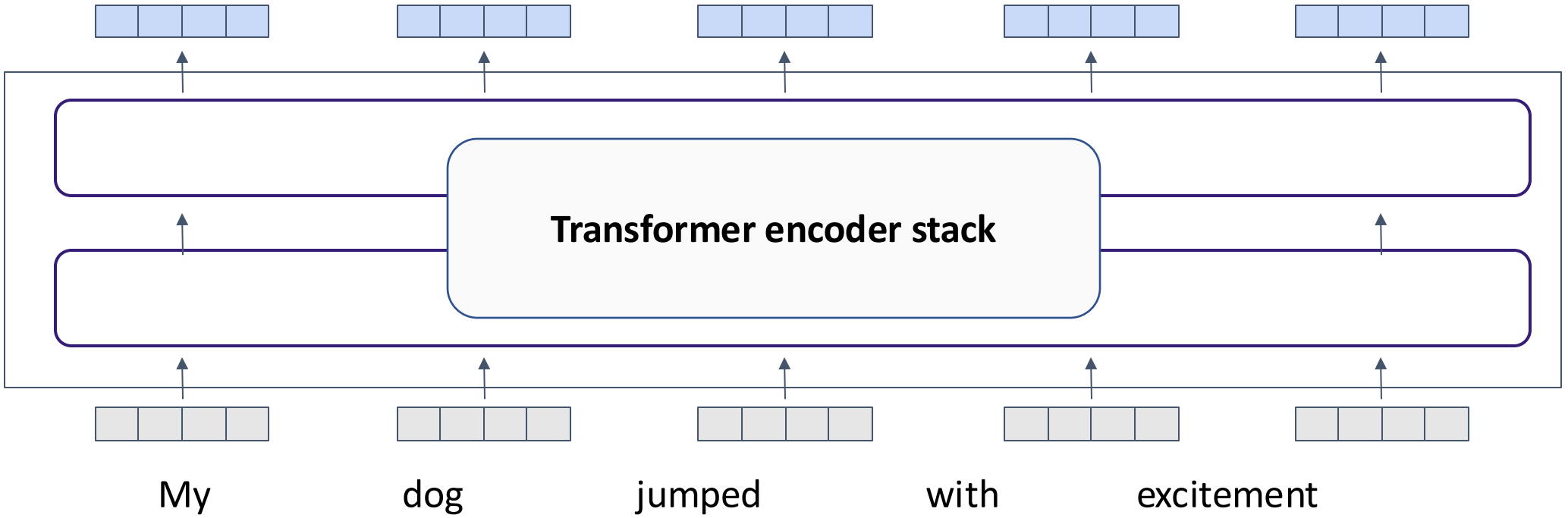


Image from [BERT: Devlin et al., 2019]

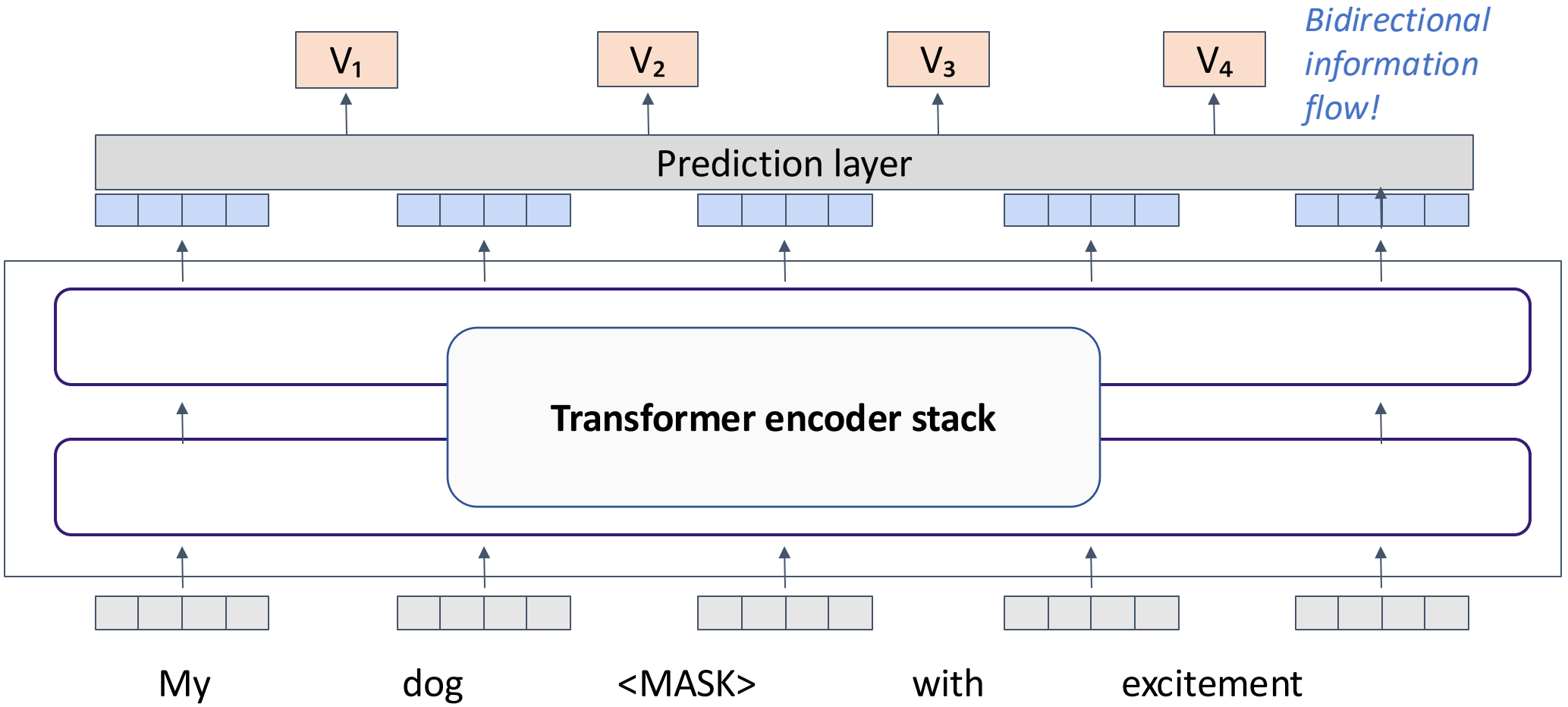
Pretraining

- Pretraining can allow learning of useful Inductive biases about the data, usually in an unsupervised way.
- Empirically, results in faster convergence and better performance on the downstream task.
- Also, usually requires much less labeled data for the downstream task.
- How to pretrain with unlabeled data?

Masked Language Model (MLM)

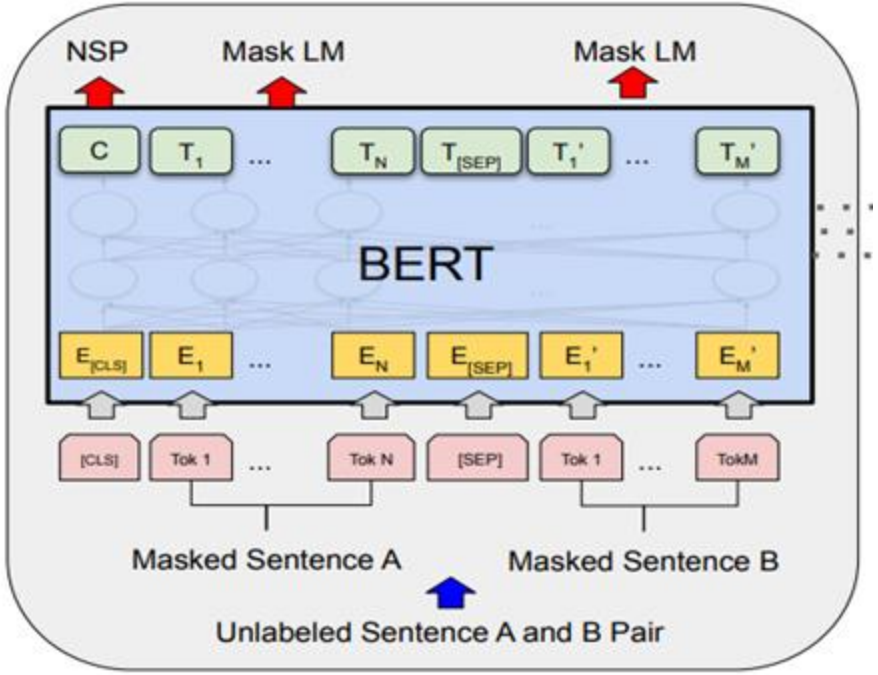


Masked Language Model (MLM)



Next Sentence Prediction (NSP)

- MLM trains model to learn relationships between tokens.
- NSP trains model to learn relationship between sentence.



Fine-Tuning

- Need small amount of data for downstream task for training.
- Only one additional (final) layer is trained for specific tasks

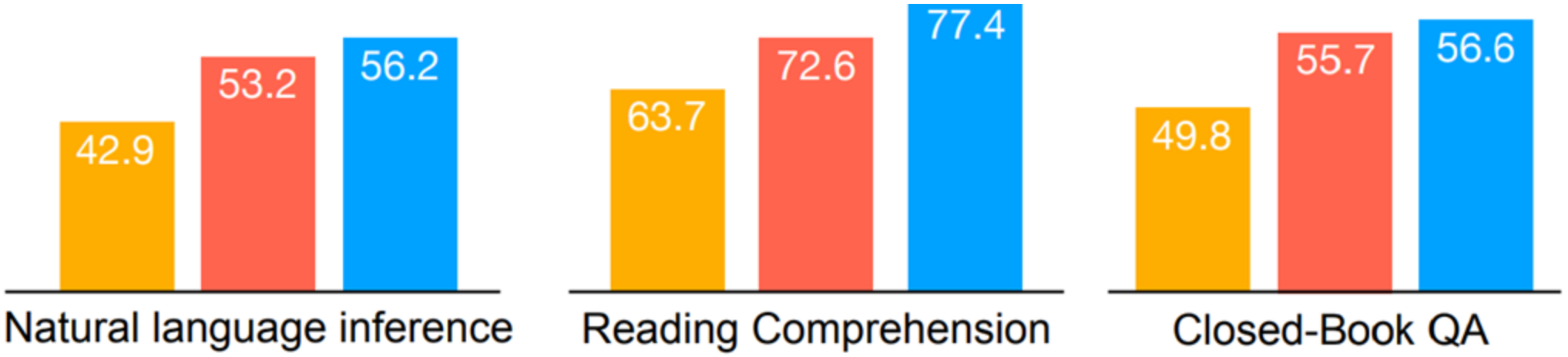


Image from Fine-Tuned Language Models Are Zero-Shot Learners

Decoder-Only Transformers

Decoder-Only Transformers

- Focus on generating output sequences
- Most suitable for language modeling tasks such as text generation and completion

Only pay attention to words already generated

- Different idea from the bidirectionality in BERT
- Use a transformer stack for language generation

How to Train a Decoder-Only Transformer Stack?

- Simply ignore attention scores for future tokens in self-attention
- Set upper-triangular portion of the QtK^T matrix o $-\infty$
- Softmax will convert these to 0 attentio scores

That is, the model will only consider previous tokens when generating the next token.

"Causal masking"

$q_1 . k_1$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
$q_2 . k_1$	$q_2 . k_2$	$-\infty$	$-\infty$	$-\infty$
$q_3 . k_1$	$q_3 . k_2$	$q_3 . k_3$	$-\infty$	$-\infty$
$q_4 . k_1$	$q_4 . k_2$	$q_4 . k_3$	$q_4 . k_4$	$-\infty$
$q_5 . k_1$	$q_5 . k_2$	$q_5 . k_3$	$q_5 . k_4$	$q_5 . k_5$

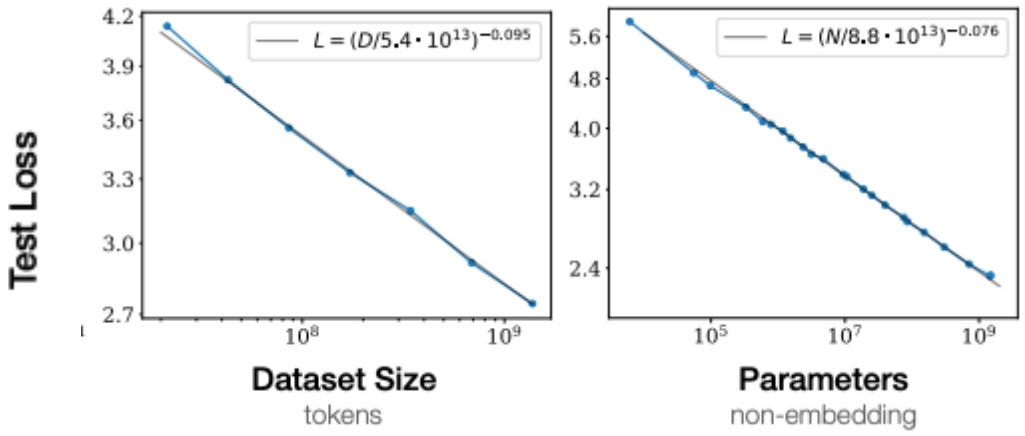
How to Train a Decoder-Only Transformer Stack? (cont.)

- Apart from causal masking, no difference from the encoder-only architecture
- ***Important:*** *Unlike RNN LMs, this training can be completely parallelized!*

$q_1 . k_1$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
$q_2 . k_1$	$q_2 . k_2$	$-\infty$	$-\infty$	$-\infty$
$q_3 . k_1$	$q_3 . k_2$	$q_3 . k_3$	$-\infty$	$-\infty$
$q_4 . k_1$	$q_4 . k_2$	$q_4 . k_3$	$q_4 . k_4$	$-\infty$
$q_5 . k_1$	$q_5 . k_2$	$q_5 . k_3$	$q_5 . k_4$	$q_5 . k_5$

Advantages of Transformer-Based Generation

- Parallelized training can enable larger models and training sets
- Training is stable compared to RNNs
  - Due to layer normalization and residual connections
  - No vanishing or exploding gradients, “path length” is always 1



Empirically, we see log-linear relationships between model size and loss, and training data size and loss.

Figure from: Scaling Laws for Neural Language Models, Kaplan et al., 2020.

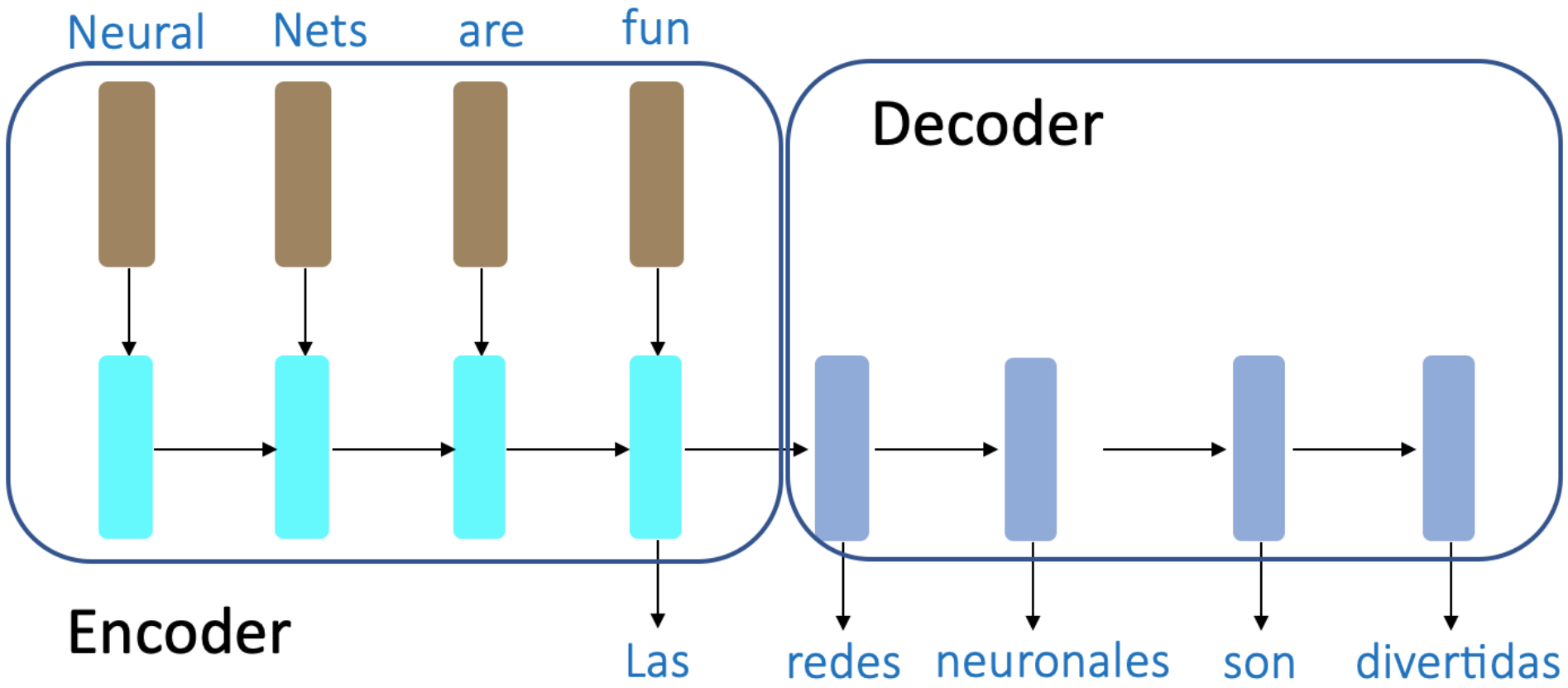
GPT Series of LMs

Model	Parameters	Decoder Layers	Context Window Size	Hidden Layer Size
GPT-1	117M	12	512	768
GPT-2	1.5B	48	1024	1600
GPT-3	175B	96	2048	12288
GPT-4	Unknown	Unknown	Unknown	Unknown

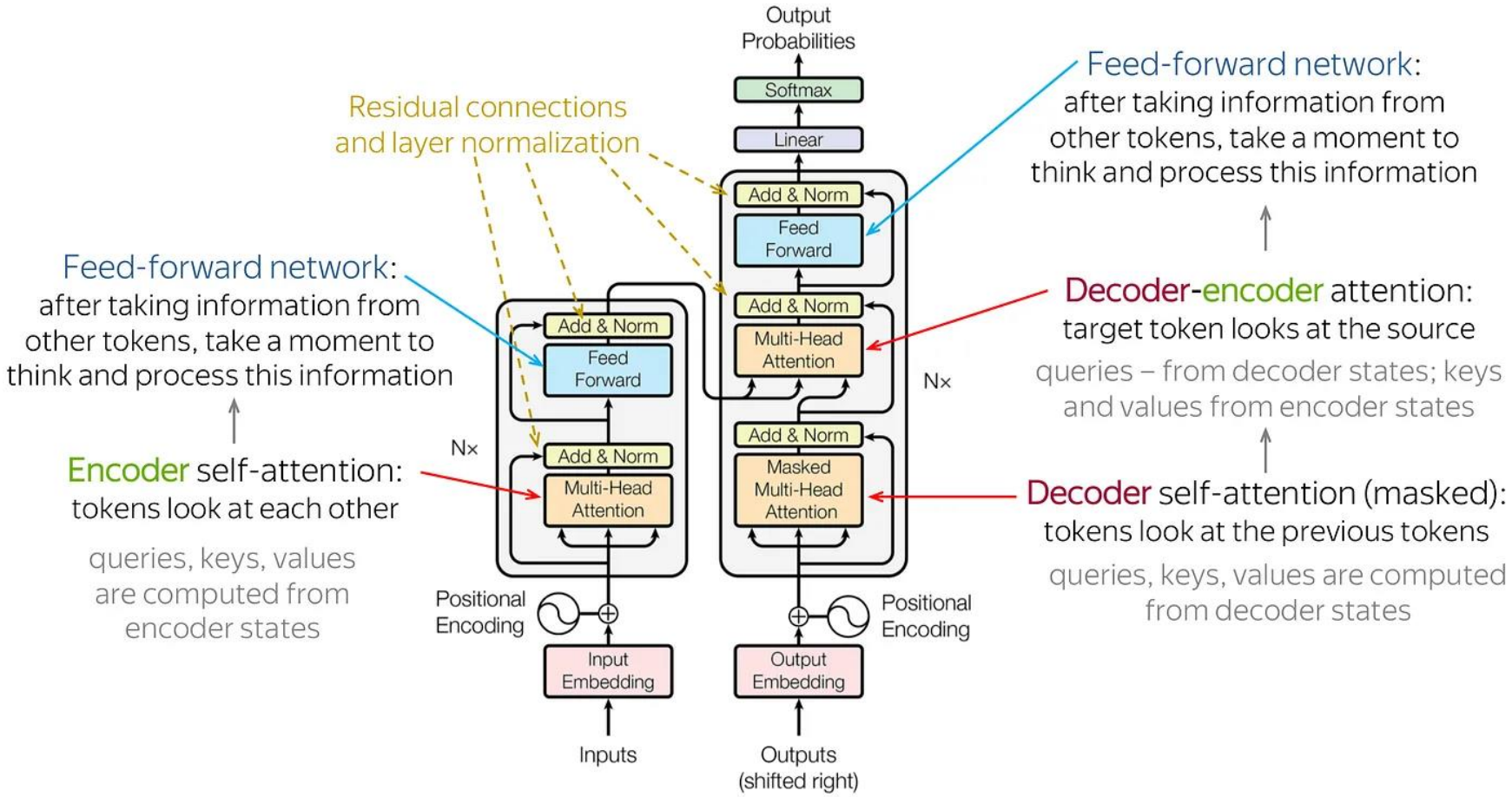
Figure from: Scaling Laws for Neural Language Models, Kaplan et al., 2020.

Encoder-Decoder Transformers

Encoder-Decoder RNNs



Encoder-Decoder Transformers



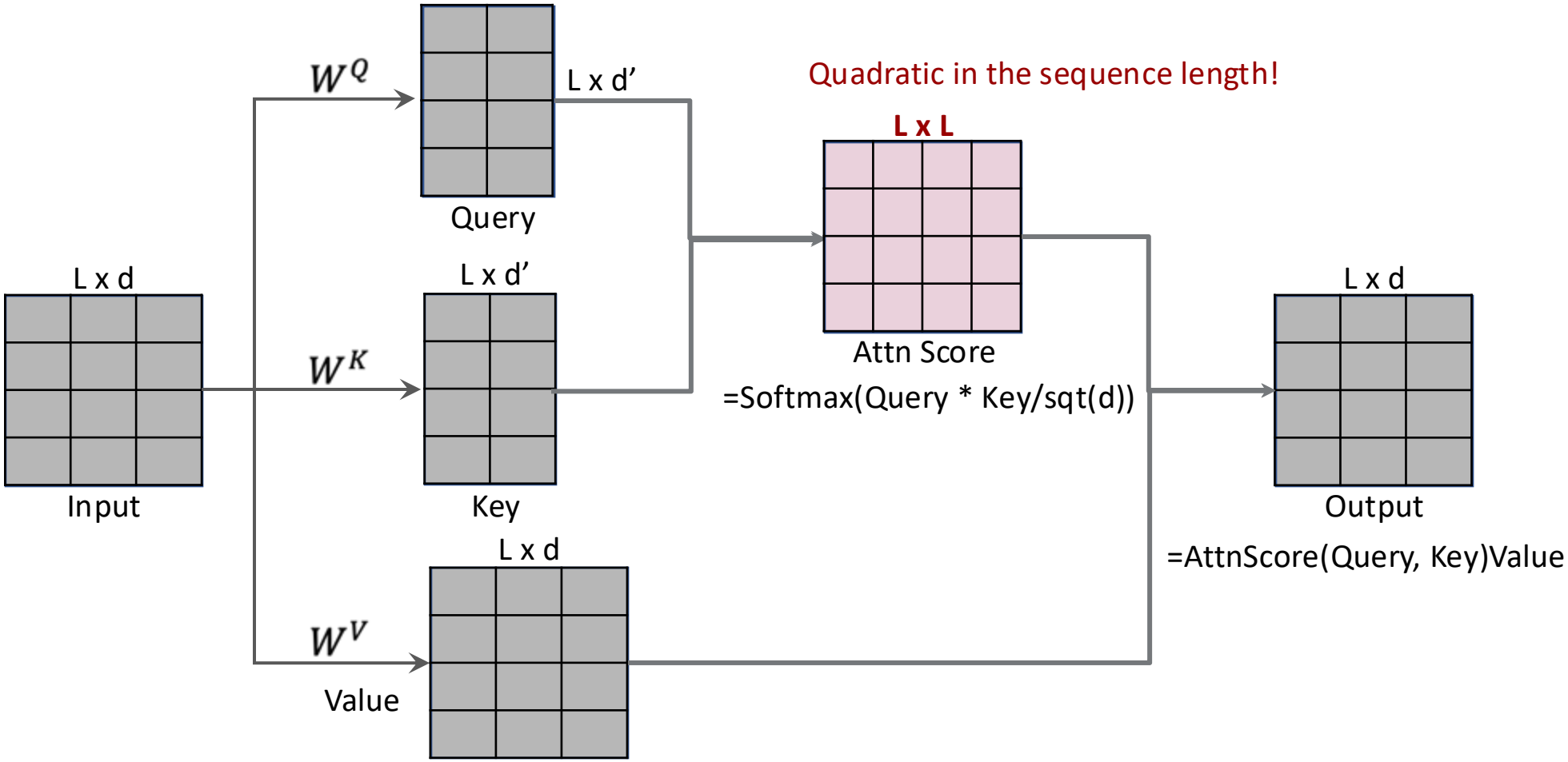
Encoder-Decoder Transformers (cont.)

- Original transformer
  - The original model introduced an encoder-decoder architecture
  - Tasks like machine translation, summarization
  - Why might this be better than decoder only?
  - *For example: Even when **he** was a young boy, George Washington …*
- T5 (Text-to-Text Transfer Transformer)
  - Unified approach where all NLP tasks are reformulated as text-to-test tasks
  - *For example: Classify sentiment: The movie was horrendous …*
- BART (Bidirectional and Auto-Regressive Transformers)
  - Combines bidirectional and auto-regressive training objective
  - Good with both generation and sequence-to-sequence tasks

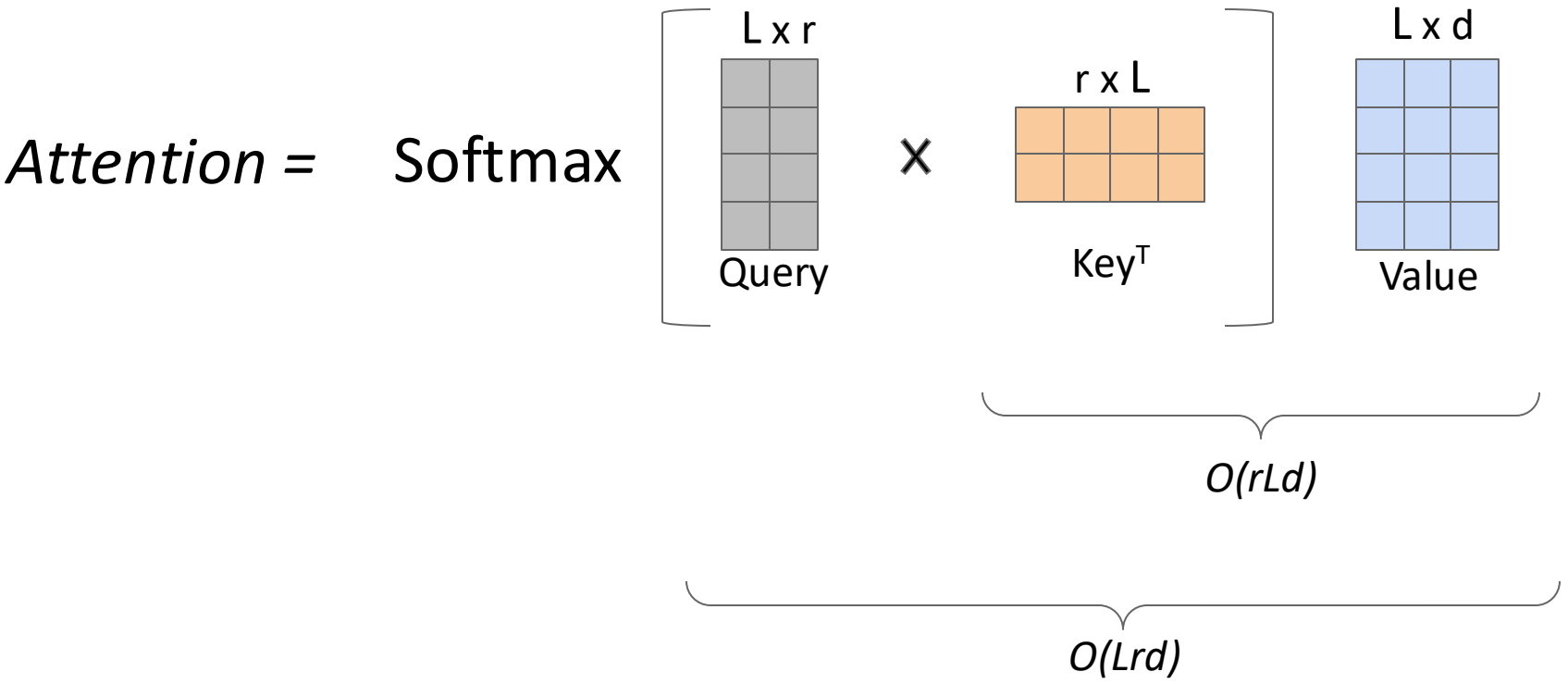
Challenges and Future Directions

- Computational issues and scalability
- Interpretability
- Domain adaptation
- …

Scalability: Computational Issue With Transformers



Scalability: Approximate Attention



Scalability: Sparse Graphs

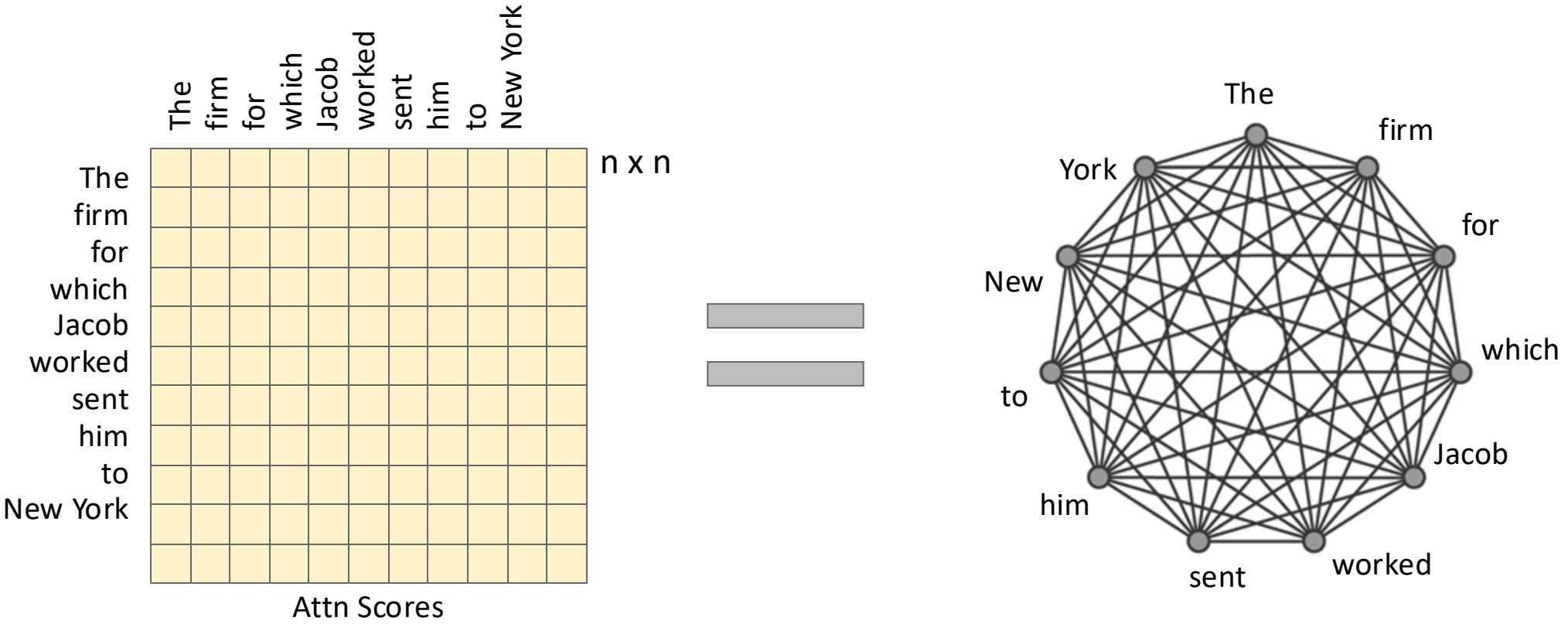


Figure from [Manzil Zaheer | Big Bird: Transformers for Longer Sequences]

Interpretability

- Decision-making processes of transformers can be unclear
- Techniques for interpreting attention weights?
- What do different heads in multi-head attention do?